

Worldline-Constrained Deterministic Substrates and Impossibility Engines

Robust Safety, Quantized Replay, and an Executable No-Collision Kernel

Preprint-ready draft | April 2026

Abstract

We develop a deterministic, constraint-first mathematical substrate in which invalid states are excluded by construction rather than detected after execution. The substrate evolves along lawful worldlines, carries intrinsic lineage and replay commitments as part of state, obeys explicit resource bounds, and preserves safety through barrier certificates. This yields a general object called an *Impossibility Engine*: an execution regime whose laws render designated forbidden outcomes unreachable. We derive a no-collision worldline kernel for multi-agent systems, prove forward invariance in discrete time, extend the analysis to continuous time with control barrier conditions, and then strengthen the framework to include bounded disturbances, robust barrier margins, estimator error tubes, quantized deterministic replay, graph-coupled swarm laws, compositional safety families, and a fail-closed executable kernel discipline. The paper closes with an implementation bridge: reference step equations, acceptance tests, and a notebook realization map that translates the mathematics directly into an executable research kernel.

Keywords: deterministic replay, control barrier functions, hybrid safety, worldline optimization, multi-agent systems, proof-carrying state, safe quadratic programming

Contents

1	Introduction and scope	2
2	Contributions of this draft	2
3	Standing assumptions and proof strategy	3
4	Formal substrate	3
4.1	Total state	3

4.2	Controls, proposals, and laws	4
4.3	Notation table	4
5	Law Projection and Executable Dynamics	5
6	Worldlines	5
7	Deterministic Lineage and Replay	6
8	Energy-Bounded Execution	6
9	Barrier Functions and Impossibility	7
10	Constraint-First Computation	8
11	Worldline Action Functional	8
12	Impossibility Engine	9
13	Discrete-Time No-Collision Worldline Kernel	10
13.1	Agent state	10
13.2	Raw dynamics	10
13.3	Pairwise safety barriers	10
13.4	Discrete barrier law	11
13.5	Mission action with hard safety	11
14	Continuous-Time Extension	12
14.1	Double-integrator swarm	12
14.2	Control barrier condition	13
15	Robust Disturbance Extension	13
15.1	Discrete disturbed law	14
15.2	Continuous disturbed law	15
16	Estimation Error Tubes and Belief-Safe Execution	15
17	Graph-Coupled Swarm Laws	16

17.1	Topology-coupled coordination objective	17
17.2	Connectivity barrier	17
18	Finite-Precision Replay and Quantized Determinism	18
18.1	Quantized state law	18
18.2	Quantization-induced barrier slack	18
19	Compositional Law Families	19
20	Hybrid Event-Triggered Worldlines	19
21	Safe Control by Projection or Quadratic Program	20
22	Latent Projection and Projected Worldline Law	21
22.1	Projected worldline prior	21
23	Master Equation	21
24	Axiom Block	22
25	Executable Kernel Specification	23
26	Notebook Realization Map	24
26.1	Reference step equations for the notebook	25
26.2	Notebook acceptance tests	25
27	Numerical Realization and Verification Protocol	26
27.1	Arithmetic law	26
27.2	Verification envelope	26
27.3	Solver discipline	26
28	Implementation Pathway to the Notebook Build	27
29	Discussion	27
30	Outlook	28

1. Introduction and scope

The objective is not merely to define another optimizer, verifier, controller, or simulation loop. The target is a computational regime with the following properties.

- P1. Deterministic replay.** Identical initial conditions and identical inputs produce identical trajectories and identical lineage.
- P2. Constraint-first execution.** Executable successor states are chosen only from a lawful admissible region.
- P3. Worldline primacy.** Trajectories, not isolated snapshots, are the fundamental computational objects.
- P4. Barrier-preserved safety.** Designated safe sets are forward invariant under lawful dynamics.
- P5. Impossibility guarantees.** Forbidden states are unreachable by law, not merely unlikely.
- P6. Bounded execution.** Every lawful transition obeys explicit local or global energy budgets.

The conceptual inversion is simple:

a transition is not valid because it happened; it happens because it is valid.

2. Contributions of this draft

This preprint-ready pass makes the framework concrete in six ways.

- C1.** It defines a deterministic, proof-carrying, constraint-first substrate whose admissible transitions are lawful by construction.
- C2.** It formalizes the *Impossibility Engine* abstraction and proves a generic forward-invariance theorem for forbidden-state exclusion.
- C3.** It derives a discrete-time and continuous-time no-collision multi-agent kernel with barrier-based safety guarantees.
- C4.** It extends the framework to bounded disturbances, estimator uncertainty tubes, graph-coupled law families, and finite-precision replay.
- C5.** It specifies an executable fail-closed kernel map together with numerical certification requirements for safe solver-mediated execution.
- C6.** It provides a direct implementation bridge to an ipynb realization via reference step equations, acceptance tests, and a staged build pathway.

3. Standing assumptions and proof strategy

Throughout the paper, the main invariance and replay results rely on a common backbone:

- A1.** the admissible transition operator is deterministic,
- A2.** the relevant barrier family is preserved or conservatively tightened under uncertainty,
- A3.** the safe projection step is feasible whenever execution is allowed to continue,
- A4.** uncertified numerical outputs are not executed and instead trigger a fail-closed response,
- A5.** replay claims are indexed by an explicit arithmetic regime.

The proof pattern is correspondingly modular. First, define a safe set by nonnegative barriers. Second, show that the lawful transition rule preserves those nonnegativity conditions. Third, lift that invariance result through uncertainty tightening, quantization margins, and solver certification. Finally, combine deterministic integration with deterministic lineage updates to obtain replayable safe execution on the chosen arithmetic lattice.

4. Formal substrate

4.1. Total state

Let the total state space be the product

$$\mathcal{X} = \mathcal{X}_{\text{phys}} \times \mathcal{X}_{\text{mem}} \times \mathcal{X}_{\text{ctrl}} \times \mathcal{X}_{\text{proof}}.$$

A state is written

$$x_t = (p_t, m_t, a_t, w_t) \in \mathcal{X},$$

where:

- p_t is physical or operational state,
- m_t is memory and commitment state,
- a_t is controller or architect state,
- w_t is witness, certificate, or proof state.

Remark 4.1. The proof-carrying component is part of the state itself. Lawfulness is internal to execution, not an external audit layer.

4.2. Controls, proposals, and laws

Let $\mathcal{U}$ denote the control space. A raw proposal map is

$$F : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}, \quad \tilde{x}_{t+1} = F(x_t, u_t).$$

The raw proposal is not automatically executable. Execution must pass through laws.

Let the law family be

$$\mathcal{C} = \{c_i : \mathcal{X} \rightarrow \mathbb{R}\}_{i \in I},$$

with equality and inequality subsets

$$I = I_{\text{eq}} \sqcup I_{\text{ineq}}.$$

The admissible region is

$$\mathcal{V} = \{x \in \mathcal{X} \mid c_i(x) = 0 \ \forall i \in I_{\text{eq}}, \ c_j(x) \leq 0 \ \forall j \in I_{\text{ineq}}\}.$$

Definition 4.2 (Admissible state). *A state $x \in \mathcal{X}$ is lawful if $x \in \mathcal{V}$.*

4.3. Notation table

Symbol	Meaning
$\mathcal{X}$	Total state space.
$\mathcal{U}$	Control or input space.
$\mathcal{V}$	Admissible lawful region.
F	Raw proposal dynamics.
Φ	Lawful transition operator.
H	Deterministic lineage or commitment update.
γ	Worldline or discrete trajectory.
Γ_{lawful}	Set of lawful worldlines.
$\mathcal{E}$	Execution or energy cost.
b	Barrier function for safe and forbidden separation.
$\mathcal{S}$	Safe set.
$\mathcal{F}$	Forbidden set.
$\Pi_{\mathcal{V}, \mathcal{L}}$	Law projection operator.
$\mathcal{A}$	Worldline action functional.
π	Latent-to-observed projection map.

5. Law Projection and Executable Dynamics

Definition 5.1 (Law projection operator). *A law projection operator is a map*

$$\Pi_{\mathcal{V}, \mathcal{L}} : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X},$$

which takes a proposal and the current state and returns a lawful successor satisfying

$$x_{t+1} = \Pi_{\mathcal{V}, \mathcal{L}}(\tilde{x}_{t+1}; x_t) \in \mathcal{V}.$$

The law projection has three canonical behaviors.

L1. Accept. If $\tilde{x}_{t+1} \in \mathcal{V}$ and lineage conditions are satisfied, then $x_{t+1} = \tilde{x}_{t+1}$.

L2. Repair. If $\tilde{x}_{t+1} \notin \mathcal{V}$, solve

$$x_{t+1} = \arg \min_{z \in \mathcal{V}} \left(d(z, \tilde{x}_{t+1}) + \lambda R_{\mathcal{L}}(z; x_t) \right),$$

where d is a state distance and $R_{\mathcal{L}}$ is a lineage repair cost.

L3. Reject or fail closed. If no acceptable repair exists, define $x_{t+1} = x_t$ or send the system to a designated safe halt state $x_{\text{halt}} \in \mathcal{V}$.

Definition 5.2 (Lawful transition operator). *The executable transition law is*

$$\Phi(x_t, u_t, \mathcal{C}, \mathcal{L}) = \Pi_{\mathcal{V}, \mathcal{L}}(F(x_t, u_t); x_t).$$

Hence the lawful dynamics are

$$x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}).$$

Axiom 5.3 (Lawfulness). *If $x_t \in \mathcal{V}$, then $x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}) \in \mathcal{V}$.*

6. Worldlines

Definition 6.1 (Worldline). *A discrete worldline of horizon T is*

$$\gamma = (x_0, x_1, \dots, x_T) \in \mathcal{X}^{T+1}.$$

Definition 6.2 (Lawful worldline). *A worldline γ is lawful if*

$$x_t \in \mathcal{V}, \quad x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}), \quad t = 0, \dots, T-1.$$

The set of lawful worldlines is

$$\Gamma_{\text{lawful}} = \left\{ \gamma \in \mathcal{X}^{T+1} \mid x_t \in \mathcal{V}, x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}) \forall t \right\}.$$

Remark 6.3. Once worldlines become primary, the substrate can optimize over histories, preserve global trajectory structure, and reason about impossibility across entire evolutions rather than only local state checks.

7. Deterministic Lineage and Replay

Let the memory state include a commitment root $h_t \in \mathcal{H}$ so that

$$m_t = (\hat{m}_t, h_t).$$

Let the lineage update law be

$$H : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \times \mathcal{H} \rightarrow \mathcal{H}, \quad h_{t+1} = H(x_t, u_t, x_{t+1}, h_t).$$

Definition 7.1 (Replay determinism). *The substrate is replay deterministic if whenever two runs satisfy*

$$x_0 = x'_0, \quad u_t = u'_t \forall t,$$

then

$$x_t = x'_t, \quad h_t = h'_t \quad \forall t.$$

Axiom 7.2 (Deterministic replay). *The lawful transition operator Φ and lineage operator H are deterministic.*

Proposition 7.3 (State-lineage uniqueness). *Under deterministic replay, the pair sequence $(x_t, h_t)_{t \geq 0}$ is uniquely determined by x_0 and the control sequence $(u_t)_{t \geq 0}$.*

Proof. The claim follows by induction. The base case is immediate from the given initial state and initial lineage. If (x_t, h_t) is fixed, then determinism of Φ yields a unique x_{t+1} and determinism of H yields a unique h_{t+1} . $\square$

8. Energy-Bounded Execution

Let the nonnegative execution cost be

$$\mathcal{E} : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}, \quad E_t = \mathcal{E}(x_t, u_t, x_{t+1}).$$

Definition 8.1 (Bounded lawful worldline). *Given either local budgets (B_t) or a global horizon*

budget B , a lawful worldline is bounded if either

$$\mathcal{E}(x_t, u_t, x_{t+1}) \leq B_t \quad \forall t,$$

or

$$\sum_{t=0}^{T-1} \mathcal{E}(x_t, u_t, x_{t+1}) \leq B.$$

The set of lawful bounded worldlines is

$$\Gamma_{\text{lawful, bounded}} = \left\{ \gamma \in \Gamma_{\text{lawful}} \left| \sum_{t=0}^{T-1} \mathcal{E}(x_t, u_t, x_{t+1}) \leq B \right. \right\}.$$

Remark 8.2. Energy or cost bounds matter because otherwise law repair can hide arbitrarily expensive search, projection, or correction behind a nominally safe transition. A bounded substrate prevents this escape hatch.

9. Barrier Functions and Impossibility

Let $\mathcal{S} \subseteq \mathcal{V}$ be a safe set and let $\mathcal{F} \subseteq \mathcal{X}$ be a forbidden set with $\mathcal{S} \cap \mathcal{F} = \emptyset$.

Definition 9.1 (Barrier function). *A barrier for $(\mathcal{S}, \mathcal{F})$ is a map $b : \mathcal{X} \rightarrow \mathbb{R}$ such that*

$$b(x) \geq 0 \quad \forall x \in \mathcal{S}, \quad b(x) < 0 \quad \forall x \in \mathcal{F}.$$

Definition 9.2 (Discrete barrier preservation). *A lawful transition preserves b with decay parameter $\alpha \in [0, 1]$ if*

$$b(x_{t+1}) - b(x_t) \geq -\alpha b(x_t),$$

that is,

$$b(x_{t+1}) \geq (1 - \alpha)b(x_t).$$

Lemma 9.3 (Barrier positivity propagates). *If $b(x_{t+1}) \geq (1 - \alpha)b(x_t)$ with $\alpha \in [0, 1]$ and $b(x_0) \geq 0$, then $b(x_t) \geq 0$ for all $t \in \mathbb{N}$.*

Proof. Because $1 - \alpha \geq 0$, the sign cannot flip from nonnegative to negative under the inequality. Induct on t . □

Theorem 9.4 (Forward invariance from barrier families). *Let*

$$\mathcal{S} = \{x \in \mathcal{V} : b_k(x) \geq 0, \quad k = 1, \dots, m\}.$$

Assume that every lawful transition satisfies

$$b_k(x_{t+1}) \geq (1 - \alpha_k)b_k(x_t), \quad \alpha_k \in [0, 1], \quad k = 1, \dots, m.$$

Then $\mathcal{S}$ is forward invariant:

$$x_0 \in \mathcal{S} \implies x_t \in \mathcal{S} \quad \forall t \in \mathbb{N}.$$

Proof. Apply the preceding lemma to each barrier b_k . Every defining inequality remains nonnegative along the lawful worldline. $\square$

Corollary 9.5 (Impossibility of forbidden states). *If $\mathcal{F} \subseteq \{x : b_k(x) < 0 \text{ for some } k\}$ and the hypotheses of the theorem hold, then every lawful worldline starting from $x_0 \in \mathcal{S}$ satisfies*

$$x_t \notin \mathcal{F} \quad \forall t \in \mathbb{N}.$$

This is the core impossibility pattern: once the lawful dynamics preserve barriers, forbidden states are unreachable.

10. Constraint-First Computation

Standard execution is often: propose, execute, detect error, repair. Here the order is inverted.

Definition 10.1 (Constraint-first transition). *A transition law is constraint-first if it is computed directly by*

$$x_{t+1} = \arg \min_{z \in \mathcal{V}(x_t)} J(x_t, u_t, z),$$

where $\mathcal{V}(x_t) \subseteq \mathcal{V}$ is the admissible successor set from x_t .

Remark 10.2. Under this formulation, invalid states are not merely penalized. They are not members of the executable successor set at all.

11. Worldline Action Functional

Define the worldline action

$$\mathcal{A}(\gamma, u) = \sum_{t=0}^{T-1} \left[L(x_t, u_t, x_{t+1}) + \sum_i \lambda_i \phi_i(x_t, x_{t+1}) \right],$$

where L is a task or transition cost and ϕ_i are law terms. For hard laws use indicator functions

$$\phi_i(x_t, x_{t+1}) = \begin{cases} 0, & \text{if the law is satisfied,} \\ +\infty, & \text{otherwise.} \end{cases}$$

Then lawful trajectory selection is

$$\gamma^* = \arg \min_{\gamma \in \Gamma_{\text{lawful}}} \mathcal{A}(\gamma, u).$$

Remark 11.1. This is where the substrate stops looking like ordinary one-step control and becomes worldline computation: the entire admissible trajectory is the primitive object.

12. Impossibility Engine

Definition 12.1 (Impossibility Engine). *An Impossibility Engine is a tuple*

$$\mathfrak{I} = (\mathcal{X}, \mathcal{U}, \mathcal{V}, \Phi, H, \mathcal{E}, \mathcal{B})$$

consisting of:

- I1. a total state space $\mathcal{X}$,*
- I2. a control space $\mathcal{U}$,*
- I3. a lawful admissible region $\mathcal{V} \subseteq \mathcal{X}$,*
- I4. a deterministic lawful transition law Φ ,*
- I5. a deterministic lineage update H ,*
- I6. a nonnegative execution cost $\mathcal{E}$,*
- I7. a family of preserved barrier certificates $\mathcal{B}$,*

with the properties:

- G1. Lawfulness:** $x_t \in \mathcal{V} \implies x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}) \in \mathcal{V}$.
- G2. Replay determinism:** identical initial states and identical controls imply identical state-lineage evolution.
- G3. Barrier preservation:** designated safe sets admit barrier families preserved by Φ .
- G4. Bounded execution:** each lawful worldline obeys an explicit budget.

Theorem 12.2 (Generic impossibility theorem). *Let $\mathfrak{I}$ be an Impossibility Engine. If $\mathcal{S} \subseteq \mathcal{V}$ is forward invariant under the preserved barrier family and $\mathcal{F} \cap \mathcal{S} = \emptyset$, then every lawful worldline with $x_0 \in \mathcal{S}$ satisfies*

$$x_t \notin \mathcal{F} \quad \forall t \in \mathbb{N}.$$

Proof. Forward invariance keeps the entire lawful worldline inside $\mathcal{S}$. Since $\mathcal{F}$ is disjoint from $\mathcal{S}$, the worldline cannot enter $\mathcal{F}$. $\square$

13. Discrete-Time No-Collision Worldline Kernel

We now specialize the substrate to a multi-agent system and make the impossibility theorem concrete.

13.1. Agent state

Let there be N agents in $\mathbb{R}^d$. For each agent $i \in \{1, \dots, N\}$ define position and velocity

$$q_t^{(i)} \in \mathbb{R}^d, \quad v_t^{(i)} \in \mathbb{R}^d.$$

Stack them into

$$q_t = (q_t^{(1)}, \dots, q_t^{(N)}) \in \mathbb{R}^{Nd}, \quad v_t = (v_t^{(1)}, \dots, v_t^{(N)}) \in \mathbb{R}^{Nd}.$$

The full substrate state may be written

$$x_t = (q_t, v_t, m_t, a_t, w_t).$$

13.2. Raw dynamics

Let the control be agent acceleration inputs

$$u_t = (u_t^{(1)}, \dots, u_t^{(N)}) \in \mathbb{R}^{Nd}.$$

Given time step $\Delta t > 0$, define the raw dynamics

$$\begin{aligned} q_{t+1}^{(i)} &= q_t^{(i)} + \Delta t v_t^{(i)}, \\ v_{t+1}^{(i)} &= v_t^{(i)} + \Delta t u_t^{(i)}. \end{aligned}$$

Collectively,

$$\tilde{x}_{t+1} = F(x_t, u_t).$$

13.3. Pairwise safety barriers

Fix pairwise safety radii $r_{ij} > 0$. Define

$$b_{ij}(q) = \|q^{(i)} - q^{(j)}\|^2 - r_{ij}^2, \quad 1 \leq i < j \leq N.$$

The safe configuration set is

$$\mathcal{S}_q = \left\{ q \in \mathbb{R}^{Nd} \mid b_{ij}(q) \geq 0 \ \forall i < j \right\}.$$

The full safe state set is

$$\mathcal{S} = \{x \in \mathcal{V} \mid q \in \mathcal{S}_q\}.$$

The forbidden collision set is

$$\mathcal{F}_{\text{coll}} = \left\{ q \in \mathbb{R}^{Nd} \mid \|q^{(i)} - q^{(j)}\| < r_{ij} \text{ for some } i < j \right\}.$$

13.4. Discrete barrier law

Assume the law projection or safe controller enforces

$$b_{ij}(q_{t+1}) \geq (1 - \alpha_{ij})b_{ij}(q_t), \quad \alpha_{ij} \in [0, 1], \quad \forall i < j.$$

Theorem 13.1 (No-collision forward invariance in discrete time). *If $x_0 \in \mathcal{S}$ and the pairwise barrier inequalities hold for every lawful transition, then*

$$b_{ij}(q_t) \geq 0 \quad \forall t \in \mathbb{N}, \forall i < j.$$

Consequently,

$$\|q_t^{(i)} - q_t^{(j)}\| \geq r_{ij} \quad \forall t \in \mathbb{N}, \forall i < j.$$

Proof. Each b_{ij} is a barrier separating the safe relation $\|q^{(i)} - q^{(j)}\| \geq r_{ij}$ from the forbidden relation $\|q^{(i)} - q^{(j)}\| < r_{ij}$. By barrier positivity propagation, each b_{ij} remains nonnegative for all time. Therefore every pairwise separation remains at least r_{ij} . $\square$

Corollary 13.2 (Collision impossibility). *Under the hypotheses of the theorem,*

$$q_t \notin \mathcal{F}_{\text{coll}} \quad \forall t \in \mathbb{N}.$$

13.5. Mission action with hard safety

Let goal positions be $g^{(i)} \in \mathbb{R}^d$. Define the goal error

$$J_{\text{goal}}(q_t) = \sum_{i=1}^N \|q_t^{(i)} - g^{(i)}\|^2.$$

Define control energy

$$\mathcal{E}(x_t, u_t, x_{t+1}) = \sum_{i=1}^N \|u_t^{(i)}\|^2.$$

A horizon action with hard safety is

$$\mathcal{A}_{\text{swarm}}(\gamma, u) = \sum_{t=0}^{T-1} \left[\sum_{i=1}^N \|u_t^{(i)}\|^2 + \beta J_{\text{goal}}(q_t) \right],$$

subject to

$$b_{ij}(q_t) \geq 0 \quad \forall t, \forall i < j.$$

Thus the executable planning problem is

$$\gamma^* = \arg \min_{\gamma \in \Gamma_{\text{lawful, bounded}}} \mathcal{A}_{\text{swarm}}(\gamma, u).$$

14. Continuous-Time Extension

The continuous-time version shows the same structure in a control-theoretic form.

14.1. Double-integrator swarm

Let agent dynamics be

$$\dot{q}^{(i)} = v^{(i)}, \quad \dot{v}^{(i)} = u^{(i)}, \quad i = 1, \dots, N.$$

Define the relative position and velocity for pair (i, j) by

$$p_{ij} = q^{(i)} - q^{(j)}, \quad w_{ij} = v^{(i)} - v^{(j)}.$$

Consider the barrier

$$h_{ij}(q) = \|p_{ij}\|^2 - r_{ij}^2.$$

Then

$$\dot{h}_{ij} = 2p_{ij}^\top w_{ij}.$$

Because the system is relative degree two with respect to u , introduce the augmented barrier quantity

$$\psi_{ij}(q, v) = \dot{h}_{ij} + \kappa_{ij} h_{ij} = 2p_{ij}^\top w_{ij} + \kappa_{ij} (\|p_{ij}\|^2 - r_{ij}^2),$$

where $\kappa_{ij} > 0$.

Its derivative is

$$\dot{\psi}_{ij} = 2\|w_{ij}\|^2 + 2p_{ij}^\top (u^{(i)} - u^{(j)}) + 2\kappa_{ij} p_{ij}^\top w_{ij}.$$

14.2. Control barrier condition

Assume the controller enforces, for some $\eta_{ij} > 0$,

$$\dot{\psi}_{ij} + \eta_{ij}\psi_{ij} \geq 0 \quad \forall i < j.$$

Equivalently,

$$2p_{ij}^\top(u^{(i)} - u^{(j)}) \geq -2\|w_{ij}\|^2 - 2\kappa_{ij}p_{ij}^\top w_{ij} - \eta_{ij}\psi_{ij}.$$

This is an affine inequality in the control inputs.

Theorem 14.1 (Continuous-time no-collision invariance). *Suppose the initial condition satisfies*

$$h_{ij}(q(0)) \geq 0, \quad \psi_{ij}(q(0), v(0)) \geq 0, \quad \forall i < j,$$

and the controller enforces

$$\dot{\psi}_{ij} + \eta_{ij}\psi_{ij} \geq 0 \quad \forall i < j.$$

Then

$$h_{ij}(q(t)) \geq 0 \quad \forall t \geq 0, \forall i < j.$$

Hence pairwise collision is impossible for all future time.

Proof. The differential inequality implies $\psi_{ij}(t) \geq 0$ for all $t \geq 0$ by Gronwall's inequality. Since

$$\dot{h}_{ij} + \kappa_{ij}h_{ij} = \psi_{ij} \geq 0,$$

we obtain another scalar comparison inequality whose solution preserves $h_{ij}(t) \geq 0$ whenever $h_{ij}(0) \geq 0$. $\square$

Remark 14.2. The continuous-time form is important because it turns impossibility into a live feasible-control condition. The system need not re-prove safety globally at each step; it only needs to maintain the barrier inequalities.

15. Robust Disturbance Extension

The previous sections assume that the raw proposal law or continuous dynamics are exact. For any real implementation, bounded uncertainty must be admitted explicitly. We therefore introduce a disturbed lawful substrate.

15.1. Discrete disturbed law

Let the proposal dynamics be perturbed by an additive disturbance δ_t :

$$\tilde{x}_{t+1} = F(x_t, u_t) + \delta_t, \quad \|\delta_t\| \leq \bar{\delta}.$$

The executable successor remains

$$x_{t+1} = \Pi_{\mathcal{V}, \mathcal{L}}(\tilde{x}_{t+1}; x_t) \in \mathcal{V}.$$

The role of robustness is to determine margins under which barrier preservation still holds despite proposal error.

Definition 15.1 (Robust barrier margin). *Let $b : \mathcal{X} \rightarrow \mathbb{R}$ be a differentiable barrier. A nonnegative scalar ρ_b is a robust barrier margin if every lawful disturbed transition satisfies*

$$b(x_{t+1}) \geq (1 - \alpha)b(x_t) - \rho_b$$

for some $\alpha \in [0, 1]$.

Lemma 15.2 (Robust positivity condition). *Suppose*

$$b(x_{t+1}) \geq (1 - \alpha)b(x_t) - \rho_b, \quad \alpha \in [0, 1], \quad \rho_b \geq 0.$$

If the initial margin satisfies

$$b(x_0) \geq \frac{\rho_b}{\alpha} \quad \text{for } \alpha > 0,$$

and if $\alpha = 0$ then $b(x_0) \geq \rho_b$, then

$$b(x_t) \geq \frac{\rho_b}{\alpha} \quad \text{for all } t$$

when $\alpha > 0$, and $b(x_t) \geq 0$ for all t when $\alpha = 0$.

Proof. For $\alpha > 0$, define $c = \rho_b/\alpha$ and $y_t = b(x_t) - c$. Then

$$y_{t+1} = b(x_{t+1}) - c \geq (1 - \alpha)b(x_t) - \rho_b - c = (1 - \alpha)(b(x_t) - c) = (1 - \alpha)y_t.$$

Hence $y_t \geq 0$ by induction whenever $y_0 \geq 0$. The case $\alpha = 0$ reduces to monotone loss bounded by ρ_b per step with the stated condition. $\square$

Theorem 15.3 (Robust forward invariance with margin). *Let*

$$\mathcal{S}_\rho = \{x \in \mathcal{V} : b_k(x) \geq \rho_k, \quad k = 1, \dots, m\}$$

for margins $\rho_k \geq \rho_{b_k}/\alpha_k$ whenever $\alpha_k > 0$. If each barrier obeys

$$b_k(x_{t+1}) \geq (1 - \alpha_k)b_k(x_t) - \rho_{b_k}$$

under all lawful disturbed transitions, then $\mathcal{S}_\rho$ is forward invariant.

Proof. Apply Lemma 10.2 to each barrier independently and intersect the resulting invariant sets. $\square$

15.2. Continuous disturbed law

For continuous-time state $x(t)$ with

$$\dot{x} = f(x) + g(x)u + w, \quad \|w\| \leq \bar{w},$$

let b be continuously differentiable. If

$$\sup_{\|w\| \leq \bar{w}} \left(\nabla b(x)^\top (f(x) + g(x)u + w) \right) \geq -\kappa b(x)$$

for $\kappa \geq 0$, then the safe set $\{x : b(x) \geq 0\}$ is forward invariant. A sufficient condition is

$$\nabla b(x)^\top (f(x) + g(x)u) \geq -\kappa b(x) + \bar{w} \|\nabla b(x)\|.$$

This is the continuous robust barrier counterpart of the discrete margin law.

16. Estimation Error Tubes and Belief-Safe Execution

In any deployed system the controller acts on an estimate rather than the true state. Let the true state be x_t and the estimate be $\hat{x}_t$, with error

$$e_t = x_t - \hat{x}_t, \quad \|e_t\| \leq \varepsilon_t.$$

The simplest safe architecture is to enforce barriers on the estimated state with a tube margin that upper bounds barrier sensitivity to state error.

Definition 16.1 (Tube-tightened barrier). *Let $b : \mathcal{X} \rightarrow \mathbb{R}$ be Lipschitz with constant L_b on the region of interest. The tube-tightened barrier is*

$$\hat{b}_t := b(\hat{x}_t) - L_b \varepsilon_t.$$

Lemma 16.2 (True-state safety from estimate safety). *If b is L_b -Lipschitz and*

$$b(\hat{x}_t) - L_b \varepsilon_t \geq 0,$$

then

$$b(x_t) \geq 0.$$

Proof. By Lipschitz continuity,

$$b(x_t) \geq b(\hat{x}_t) - L_b \|x_t - \hat{x}_t\| \geq b(\hat{x}_t) - L_b \varepsilon_t \geq 0.$$

□

Definition 16.3 (Error tube). *An error tube is a sequence of sets $\mathcal{T}_t \subseteq \mathcal{X}$ such that*

$$x_t \in \hat{x}_t \oplus \mathcal{T}_t \quad \forall t,$$

where $\oplus$ denotes Minkowski addition.

Theorem 16.4 (Belief-safe forward invariance). *Assume each barrier b_k is Lipschitz with constant L_k on the relevant tube, and that the estimated evolution satisfies*

$$b_k(\hat{x}_{t+1}) - L_k \varepsilon_{t+1} \geq (1 - \alpha_k)(b_k(\hat{x}_t) - L_k \varepsilon_t).$$

If

$$b_k(\hat{x}_0) - L_k \varepsilon_0 \geq 0 \quad \forall k,$$

then the true state remains in the safe set for all time.

Proof. The tightened quantity obeys the same positivity-preserving recursion as in the nominal discrete barrier theorem. Lemma 11.2 then transfers the result from estimated to true state. □

Remark 16.5. This is the correct bridge from abstract lawful dynamics to systems with sensors, estimators, and delayed or noisy observations. The controller can operate on certified belief tubes rather than assuming exact state access.

17. Graph-Coupled Swarm Laws

The no-collision swarm law becomes more realistic once communication topology is modeled explicitly. Let $G_t = (V, E_t)$ be a time-varying interaction graph on agents $V = \{1, \dots, N\}$. Let $A_t = [a_{ij}(t)]$ be a symmetric weighted adjacency matrix and let $L_t = D_t - A_t$ be the graph Laplacian.

17.1. Topology-coupled coordination objective

For agent positions $q_t^{(i)} \in \mathbb{R}^d$, define the graph smoothness penalty

$$\Psi_G(q_t) = \frac{1}{2} \sum_{(i,j) \in E_t} a_{ij}(t) \|q_t^{(i)} - q_t^{(j)}\|^2 = q_t^\top (L_t \otimes I_d) q_t.$$

This term penalizes disagreement across the active communication graph and can be combined with mission objectives and hard safety barriers.

Definition 17.1 (Graph-lawful worldline). *A swarm worldline is graph-lawful if, in addition to state admissibility and barrier preservation, the graph process G_t obeys a law family such as connectivity, bounded packet delay, or minimum dwell time between edge changes.*

17.2. Connectivity barrier

Let $\lambda_2(L_t)$ denote the algebraic connectivity of the current graph. A communication safety condition is

$$\lambda_2(L_t) \geq \underline{\lambda} > 0.$$

Define a graph barrier

$$b_G(G_t) = \lambda_2(L_t) - \underline{\lambda}.$$

If the graph update law satisfies

$$b_G(G_{t+1}) \geq (1 - \alpha_G) b_G(G_t),$$

then graph connectivity is forward invariant.

Theorem 17.2 (Joint physical-communication safety). *Suppose pairwise collision barriers b_{ij} and the graph barrier b_G are all preserved by the lawful transition operator. Then the intersection set*

$$\mathcal{S}_{\text{joint}} = \{x \in \mathcal{V} : b_{ij}(x) \geq 0 \ \forall i < j, \ b_G(x) \geq 0\}$$

is forward invariant.

Proof. This is an immediate application of the multi-barrier forward invariance theorem on the product law family consisting of physical barriers and topology barriers. $\square$

Remark 17.3. The graph itself becomes part of the lawful state, not a side channel. This is essential for swarm systems whose coordination guarantees depend as much on communications as on geometry.

18. Finite-Precision Replay and Quantized Determinism

Real machines do not execute over exact reals. If replay determinism matters, the substrate must specify a lawful arithmetic model.

18.1. Quantized state law

Let $Q_\eta : \mathcal{X} \rightarrow \mathcal{X}_\eta$ be a quantizer at resolution $\eta > 0$. Define the executable machine law by

$$x_{t+1}^{(\eta)} = Q_\eta\left(\Phi(x_t^{(\eta)}, u_t, \mathcal{C}, \mathcal{L})\right).$$

The corresponding lineage update is

$$h_{t+1}^{(\eta)} = H_\eta(x_t^{(\eta)}, u_t, x_{t+1}^{(\eta)}, h_t^{(\eta)}),$$

where H_η is deterministic over quantized representations.

Definition 18.1 (Quantized replay determinism). *A substrate is quantized replay deterministic if identical quantized initial conditions and identical inputs imply identical quantized state-lineage sequences.*

Proposition 18.2 (Machine-level replay determinism). *If Q_η , Φ , and H_η are deterministic functions on the machine representation, then the quantized substrate is replay deterministic.*

Proof. This follows immediately by induction on time because the next quantized state and next lineage are deterministic functions of the previous quantized state and control. $\square$

18.2. Quantization-induced barrier slack

Assume b is Lipschitz with constant L_b and the quantizer obeys

$$\|Q_\eta(x) - x\| \leq c_Q \eta.$$

Then any barrier evaluated on the quantized state incurs at most slack

$$|b(Q_\eta(x)) - b(x)| \leq L_b c_Q \eta.$$

Therefore safe execution on a digital machine must tighten barriers by at least this amount.

Theorem 18.3 (Quantized safety tightening). *If the machine enforces*

$$b(x_t^{(\eta)}) \geq L_b c_Q \eta \quad \forall t,$$

then the corresponding ideal state represented by $x_t^{(\eta)}$ satisfies $b(x_t) \geq 0$.

Proof. By the displayed Lipschitz bound,

$$b(x_t) \geq b(x_t^{(\eta)}) - L_b c_Q \eta \geq 0.$$

□

19. Compositional Law Families

A practical substrate will never run with only one safety family. It must simultaneously preserve physical separation, communication connectivity, resource budgets, and lineage consistency. We therefore formalize composition.

Definition 19.1 (Composed lawful set). *Let $\mathcal{V}^{(1)}, \dots, \mathcal{V}^{(r)} \subseteq \mathcal{X}$ be lawful regions associated with distinct law families. The composed lawful set is*

$$\mathcal{V}^{\text{comp}} = \bigcap_{\ell=1}^r \mathcal{V}^{(\ell)}.$$

A transition operator is compositional if it maps $\mathcal{V}^{\text{comp}}$ into itself.

Theorem 19.2 (Compositional forward invariance). *Suppose for each law family $\ell \in \{1, \dots, r\}$ there exists a barrier family $\{b_{\ell,m}\}_m$ whose nonnegativity characterizes $\mathcal{V}^{(\ell)}$, and suppose each barrier is preserved by the common lawful transition operator Φ . Then the composed lawful set $\mathcal{V}^{\text{comp}}$ is forward invariant.*

Proof. Every individual family is forward invariant under Φ . The intersection of forward-invariant sets is forward invariant. □

Corollary 19.3 (Simultaneous impossibility). *If forbidden events are partitioned across law families, for example*

$$\mathcal{F} = \mathcal{F}_{\text{coll}} \cup \mathcal{F}_{\text{disc}} \cup \mathcal{F}_{\text{budget}} \cup \mathcal{F}_{\text{lineage}},$$

and each component forbidden set lies outside its corresponding invariant lawful set, then no lawful trajectory beginning in $\mathcal{V}^{\text{comp}}$ can enter $\mathcal{F}$.

20. Hybrid Event-Triggered Worldlines

Many real systems are neither purely discrete-time nor purely continuous-time. They evolve continuously between event times and undergo lawful resets at triggering instants. Let $\{\tau_k\}_{k \geq 0}$ be an increasing event sequence. Between events,

$$\dot{x} = f(x) + g(x)u, \quad t \in (\tau_k, \tau_{k+1}),$$

and at events,

$$x(\tau_k^+) = R(x(\tau_k^-), \sigma_k),$$

where R is a lawful reset map parameterized by an event symbol σ_k .

Definition 20.1 (Hybrid lawful execution). *A hybrid execution is lawful if the flow phase preserves all continuous barrier conditions and every reset map R maps the safe set into itself.*

Theorem 20.2 (Hybrid forward invariance). *Assume:*

- (i) *continuous flow preserves every barrier in the sense of the continuous-time barrier condition,*
- (ii) *every reset satisfies $b(R(x, \sigma)) \geq 0$ whenever $b(x) \geq 0$,*
- (iii) *event times are non-accumulating.*

Then the safe set is forward invariant for the hybrid execution.

Proof. Safety is preserved during each flow interval by the continuous barrier condition, and at each jump by the reset condition. Since event times do not accumulate, every finite horizon contains finitely many jumps, so induction across jumps completes the proof. $\square$

21. Safe Control by Projection or Quadratic Program

The previous section yields a practical safe control synthesis pattern. Let u_{nom} be any nominal control from a planner. Define the safe control at each instant by the quadratic program

$$u^* = \arg \min_{u \in \mathbb{R}^{N_d}} \frac{1}{2} \|u - u_{\text{nom}}\|^2$$

subject to all affine barrier constraints

$$2p_{ij}^\top(u^{(i)} - u^{(j)}) \geq -2\|w_{ij}\|^2 - 2\kappa_{ij}p_{ij}^\top w_{ij} - \eta_{ij}\psi_{ij}, \quad \forall i < j.$$

Proposition 21.1 (Safe projection interpretation). *When the barrier-feasible set of controls is nonempty, the quadratic program above returns the Euclidean projection of the nominal control onto the barrier-feasible control polyhedron.*

Proof. The objective is strictly convex and the feasible region is an intersection of affine half-spaces. Therefore the optimizer is unique and equals the Euclidean projection onto that closed convex set. $\square$

This is the continuous-time analog of law projection. The controller is projected into the admissible set before execution.

22. Latent Projection and Projected Worldline Law

To formalize hidden structure without collapsing into vague metaphysics, introduce a latent state space $\mathcal{Z}$ and a projection

$$\pi : \mathcal{Z} \rightarrow \mathcal{X}.$$

Let latent dynamics evolve as

$$z_{t+1} = G(z_t, u_t), \quad x_t = \pi(z_t).$$

The projected worldline is lawful when

$$\pi(z_t) \in \mathcal{V}, \quad \pi(z_{t+1}) = \Phi(\pi(z_t), u_t, \mathcal{C}, \mathcal{L}).$$

Definition 22.1 (Projected lawful manifold). *A coupled manifold $\mathcal{M} \subseteq \mathcal{Z} \times \mathcal{X}$ is lawful if*

$$(z_t, \pi(z_t)) \in \mathcal{M} \quad \forall t,$$

and the projected observable worldline lies in Γ_{lawful} .

22.1. Projected worldline prior

Suppose one observable coordinate y_t should approximately follow a projected parabolic prior

$$y_t \approx y_0 + v_0 t - \frac{1}{2} g t^2.$$

Define the regularizer

$$R_{\text{proj}}(\gamma) = \sum_{t=0}^T \left(y_t - y_0 - v_0 t + \frac{1}{2} g t^2 \right)^2.$$

Then the total action may be augmented as

$$\mathcal{A}_{\text{total}}(\gamma, u) = \mathcal{A}(\gamma, u) + \mu R_{\text{proj}}(\gamma), \quad \mu \geq 0.$$

Remark 22.2. This does not require claiming literal gravity. It encodes a lawful projected trajectory prior in the observable evolution.

23. Master Equation

The substrate can be written compactly as

$$x_{t+1} = \arg \min_z [J(x_t, u_t, z) + \mu \mathcal{E}(x_t, u_t, z) + \nu R_{\mathcal{L}}(z; x_t) + \eta R_{\text{proj}}(\gamma_{0:t} \oplus z)]$$

subject to

$$c_i(z) = 0, \quad c_j(z) \leq 0, \quad b_k(z) \geq 0,$$

with lineage update

$$h_{t+1} = H(x_t, u_t, x_{t+1}, h_t).$$

The terms have the following roles.

- J is the task objective.
- $\mathcal{E}$ is the execution cost or energy budget term.
- $R_{\mathcal{L}}$ is lineage consistency or repair cost.
- R_{proj} is the projected worldline prior.
- c_i, c_j define hard equalities and inequalities.
- b_k define barrier-preserved safety constraints.

This equation is the compact signature of the substrate: lawful trajectory-native execution with embedded proof, bounded cost, and barrier-preserved impossibility.

24. Axiom Block

For concise reference, the framework reduces to the following axioms.

Axiom 24.1 (State completeness). *All physical, memory, control, and witness variables are contained in the total state $x_t \in \mathcal{X}$.*

Axiom 24.2 (Admissibility). *Only states in $\mathcal{V}$ are executable.*

Axiom 24.3 (Lawful transition). *Every executable transition is produced by the deterministic operator*

$$x_{t+1} = \Phi(x_t, u_t, \mathcal{C}, \mathcal{L}) \in \mathcal{V}.$$

Axiom 24.4 (Intrinsic lineage). *Every lawful transition updates a commitment or witness state by the deterministic rule*

$$h_{t+1} = H(x_t, u_t, x_{t+1}, h_t).$$

Axiom 24.5 (Barrier preservation). *Every designated safe barrier remains nonnegative under lawful transitions.*

Axiom 24.6 (Bounded execution). *Every lawful worldline obeys an explicit resource or energy budget.*

Axiom 24.7 (Worldline primacy). *Optimization and verification are defined over trajectories, not isolated states.*

25. Executable Kernel Specification

The doctrine is now strong enough to collapse into an executable reference kernel. Define the per-step kernel map

$$\mathcal{K} : (\hat{x}_t, \rho_t, u_{\text{nom},t}, h_t) \mapsto (x_{t+1}, \hat{x}_{t+1}, u_t^*, h_{t+1}),$$

where:

- $\hat{x}_t$ is the estimator state,
- ρ_t is the uncertainty descriptor (tube radius, covariance surrogate, or interval witness),
- $u_{\text{nom},t}$ is the nominal task control,
- u_t^* is the law-projected safe control,
- h_t is the deterministic lineage state.

A single kernel step is specified by the following deterministic stages:

K1. Estimate. Compute $\hat{x}_t$ and uncertainty descriptor ρ_t from the prior state and observation stream.

K2. Tighten. For each barrier b_k , compute a conservative margin $\Delta_k(\rho_t)$ and tightened barrier

$$\tilde{b}_k(\hat{x}_t) = b_k(\hat{x}_t) - \Delta_k(\rho_t).$$

K3. Plan. Generate a nominal control $u_{\text{nom},t}$ from the task objective or worldline planner.

K4. Project. Solve the safe quadratic program or projection law to obtain u_t^* in the barrier-feasible set.

K5. Advance. Update the physical and memory state using the deterministic integration rule

$$x_{t+1} = \Phi(x_t, u_t^*, \mathcal{C}, \mathcal{L}), \quad h_{t+1} = H(x_t, u_t^*, x_{t+1}, h_t).$$

K6. Verify. Recompute the witnesses, commitments, and barrier residuals; if any verification fails, halt into the designated safe state.

This yields an implementation-facing interpretation of law projection: the kernel never emits raw controls or raw states directly to execution. It emits only the projected, certified, lineage-updated successor.

Theorem 25.1 (Executable kernel inherits safety and replay). *Suppose the following conditions hold for every step of $\mathcal{K}$:*

- E1. the tightening margins $\Delta_k(\rho_t)$ upper-bound the combined effect of estimation error, bounded disturbance, and quantization error on barrier b_k ,*
- E2. the safe projection or quadratic program is feasible,*
- E3. the state integrator, quantizer, and lineage update are deterministic,*
- E4. the initial state lies in the tightened safe set.*

Then the executable kernel trajectory is forward invariant in the true safe set, and the emitted state-lineage sequence is replay deterministic on the chosen arithmetic lattice.

Proof. The tightened barriers reduce the uncertain system to the nominal preserved-barrier form treated in the robust, belief-safe, and quantized invariance theorems above. Feasibility of the projection step ensures every emitted control lies in the admissible control set. Deterministic integration, quantization, and lineage update then imply replay determinism by the replay axioms. Therefore every kernel-emitted successor is lawful, safe, and reproducible. $\square$

26. Notebook Realization Map

The clearest implementation bridge is a research notebook whose cells follow the law structure one-to-one. A minimal but complete notebook realization can be organized into the following modules:

- N1. Geometry and state module.** Define agent state tensors, graph data, pairwise offsets, velocity stacks, and the deterministic lattice or fixed-step integrator.
- N2. Barrier module.** Implement b_{ij} , robust margins, topology barriers, and their Jacobians with respect to the control variables.
- N3. Planner module.** Produce u_{nom} from waypoint goals, formation objectives, or a horizon worldline action.
- N4. Safety projection module.** Solve the safe quadratic program and return u^* together with residual certificates and active-constraint information.
- N5. Replay and lineage module.** Hash or commit the tuple (x_t, u_t^*, x_{t+1}) so every run is reproducible and auditable.
- N6. Simulation module.** Advance the deterministic state, apply disturbance models when desired, and update the estimator and belief tube.
- N7. Verification module.** Assert invariants at every step: separation, connectivity, budget, and commitment continuity.

N8. Visualization module. Render trajectories, barrier values, active constraints, graph edges, budget traces, and replay comparisons.

A notebook implementing these modules is not a separate informal demo. It is a direct executable image of the paper. The key rule is that every plotted or logged quantity should correspond to a mathematical object already defined in this document.

26.1. Reference step equations for the notebook

For a reference discrete implementation, the notebook can use the following per-step sequence:

$$\begin{aligned}\hat{x}_t &= \mathcal{O}(y_{0:t}, u_{0:t-1}), \\ u_{\text{nom},t} &= \mathcal{P}(\hat{x}_t, g_t), \\ u_t^* &= \text{Proj}_{\mathcal{U}_{\text{safe}}(\hat{x}_t, \rho_t)}(u_{\text{nom},t}), \\ x_{t+1} &= \Phi(x_t, u_t^*, \mathcal{C}, \mathcal{L}), \\ h_{t+1} &= H(x_t, u_t^*, x_{t+1}, h_t), \\ \rho_{t+1} &= \mathcal{T}(\rho_t, x_t, u_t^*, y_{t+1}).\end{aligned}$$

Here $\mathcal{O}$ is the estimator, $\mathcal{P}$ is the nominal planner, and $\mathcal{T}$ is the uncertainty-tube update law.

26.2. Notebook acceptance tests

Before calling the implementation complete, the notebook should pass at least the following tests:

- A1. Replay test:** rerunning from the same initial seed and same arithmetic mode produces identical state and commitment traces,
- A2. Safety test:** no pairwise barrier becomes negative,
- A3. Robustness test:** bounded disturbances inside the assumed tube do not violate the true safe set,
- A4. Quantization test:** the tightened margins are sufficient for the selected precision,
- A5. Compositional test:** physical barriers and graph-connectivity barriers remain simultaneously satisfied,
- A6. Stress test:** active constraints switch on and off without breaking feasibility of the safe projection step.

27. Numerical Realization and Verification Protocol

A mathematically serious implementation must separate theorem assumptions from numerical artifacts. The following protocol closes that gap.

27.1. Arithmetic law

Fix an arithmetic regime $\mathfrak{a}$, for example IEEE floating point with a declared tolerance or a fixed-point lattice with declared quantizer Q . Every reported replay claim must be indexed by that regime. In particular, replay means

$$(x_0, u_{0:T-1}, \mathfrak{a}) = (x'_0, u'_{0:T-1}, \mathfrak{a}') \implies (x_{0:T}, h_{0:T}) = (x'_{0:T}, h'_{0:T})$$

only when the arithmetic regime is the same.

27.2. Verification envelope

Let ε_{num} denote the certified numerical error budget induced by integration, solver tolerance, and quantization. Then every practical barrier check should be carried out on the tightened quantity

$$b_k^{\text{num}}(x) = b_k(x) - \varepsilon_{\text{num},k}.$$

This aligns the implementation with the quantized safety theorem rather than silently assuming exact arithmetic.

27.3. Solver discipline

The safety projection layer should record, at every step, the optimization status, primal residual, dual residual, and the identity of active barrier constraints. A step may be executed only if these diagnostics lie inside a declared certification envelope; otherwise the kernel should fail closed.

Proposition 27.1 (Fail-closed numerical safety discipline). *Assume the safe projection solver returns either a certified feasible solution inside the declared residual envelope or an explicit failure flag, and assume the kernel executes a control only in the certified-feasible case. Then numerical solver failure cannot produce an unsafe executed control; it can only trigger a safe halt or hold state.*

Proof. By construction, uncertified solver outputs are never passed to the dynamics. Therefore the only controls that reach execution are either barrier-feasible certified controls or the predefined safe halt action. Unsafe uncertified controls are excluded from the executed control alphabet. $\square$

28. Implementation Pathway to the Notebook Build

The right build sequence is therefore not exploratory sprawl but a narrow implementation ladder:

P1. implement the deterministic state and barrier geometry,

P2. implement the nominal planner,

P3. implement the safe quadratic program,

P4. implement lineage and replay hashing,

P5. implement robust margins and estimator tubes,

P6. add graph-coupled laws,

P7. add stress tests and visual diagnostics,

P8. only then add richer worldline optimization horizons.

This order matters. If the notebook begins with long-horizon fancy planners before deterministic replay and barrier correctness are locked, the implementation will be visually impressive but mathematically unreliable. The correct build starts with lawfulness, then replay, then safety projection, then robustness, and only then richer autonomy.

29. Discussion

The resulting object is not merely a controller, not merely a verifier, and not merely a simulation kernel. It is a law-governed execution regime in which:

- state is proof-carrying,
- transition is admissibility-constrained,
- history is deterministic and replayable,
- safety is forward invariant,
- forbidden outcomes are structurally excluded,
- trajectories are the native objects of optimization,
- uncertainty, estimation error, and finite-precision execution are incorporated as first-class mathematical laws.

That is the mathematical core of the framework in a form closer to implementable kernels rather than high-level doctrine.

30. Outlook

The strongest next pass after this one is no longer to broaden the doctrine but to deepen a single specialization into a theorem-heavy technical paper. The cleanest path is a dedicated *Robust No-Collision Worldline Kernel* document with:

- R1.** explicit feasibility conditions for the safe quadratic program,
- R2.** delayed and packet-dropped graph dynamics,
- R3.** actuator saturation and reachable-set bounds,
- R4.** convergence guarantees to task goals under active barrier constraints,
- R5.** tube MPC or receding-horizon variants of the worldline action,
- R6.** compositional proofs for collision safety, connectivity, budget, and lineage in a single hybrid theorem stack.

That is the right next move if the objective is to turn the substrate into a mathematically serious autonomous kernel.